DATA STRUCTURES AND ALGORITHMS

C Programming Practical Solutions

Question Bank Solutions | Questions 3–41

Prepared according to the uploaded FCRIT DSA Practical Program List and the Semester III Data Structures syllabus.

How to use this booklet

Each question is provided as a separate C program. Compile and run one program at a time. Programs use standard C input/output, structures, pointers, dynamic memory allocation, arrays, recursion, and functions.

Compilation

```
gcc filename.c -o program
./program
```

On Windows with MinGW: gcc filename.c -o program.exe, then run program.exe.

Q3. Static Stack ADT using an Array

```
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;

void push(void) {
    int x;
    if (top == MAX - 1) { printf("Stack Overflow\n"); return; }
    printf("Enter element: "); scanf("%d", &x);
    stack[++top] = x;
}
void pop(void) {
    if (top == -1) printf("Stack Underflow\n");
    else printf("Deleted: %d\n", stack[top--]);
}
void peek(void) {
    if (top == -1) printf("Stack is empty\n");
    else printf("Top: %d\n", stack[top]);
}
void display(void) {
    int i;
    if (top == -1) { printf("Stack is empty\n"); return; }
    for (i = top; i >= 0; i--) printf("%d\n", stack[i]);
}
int main(void) {
    int ch;
    do {
        printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: break;
            default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}
```

Q4. Dynamic Stack ADT using a Resizable Array

The array grows using realloc() when it becomes full.

```
#include <stdio.h>
#include <stdlib.h>
int *stack = NULL, top = -1, capacity = 2;

void push(void) {
    int x, *temp;
    printf("Enter element: "); scanf("%d", &x);
    if (top == capacity - 1) {
        capacity *= 2;
        temp = (int *)realloc(stack, capacity * sizeof(int));
        if (temp == NULL) { printf("Memory allocation failed\n"); capacity /= 2; return; }
        stack = temp;
    }
    stack[++top] = x;
}
void pop(void) {
    if (top == -1) printf("Stack Underflow\n");
    else printf("Deleted: %d\n", stack[top--]);
}
void peek(void) {
    if (top == -1) printf("Stack is empty\n");
    else printf("Top: %d\n", stack[top]);
}
void display(void) {
    int i;
    if (top == -1) { printf("Stack is empty\n"); return; }
    for (i = top; i >= 0; i--) printf("%d\n", stack[i]);
}
int main(void) {
    int ch;
    stack = (int *)malloc(capacity * sizeof(int));
    if (stack == NULL) return 1;
    do {
        printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
```

```
            case 4: display(); break;
            case 5: break;
            default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    free(stack);
    return 0;
}
```

Q5. Dynamic Stack ADT using a Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *top = NULL;

void push(void) {
    int x; struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    if (n == NULL) { printf("Memory allocation failed\n"); return; }
    printf("Enter element: "); scanf("%d", &x);
    n->data = x; n->next = top; top = n;
}
void pop(void) {
    struct Node *t;
    if (top == NULL) { printf("Stack Underflow\n"); return; }
    t = top; printf("Deleted: %d\n", t->data);
    top = top->next; free(t);
}
void peek(void) {
    if (top == NULL) printf("Stack is empty\n");
    else printf("Top: %d\n", top->data);
}
void display(void) {
    struct Node *t = top;
    if (t == NULL) { printf("Stack is empty\n"); return; }
    while (t != NULL) { printf("%d\n", t->data); t = t->next; }
}
int main(void) {
    int ch;
    do {
        printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break; case 2: pop(); break;
            case 3: peek(); break; case 4: display(); break;
            case 5: break; default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    while (top != NULL) { struct Node *t = top; top = top->next; free(t); }
    return 0;
}
```

Q6. Static Queue using an Array

```
#include <stdio.h>
#define MAX 5
int q[MAX], front = -1, rear = -1;
void insert(void) {
    int x;
    if (rear == MAX - 1) { printf("Queue Overflow\n"); return; }
    printf("Enter element: "); scanf("%d", &x);
    if (front == -1) front = 0;
    q[++rear] = x;
}
void deleteItem(void) {
    if (front == -1 || front > rear) { printf("Queue Underflow\n"); return; }
    printf("Deleted: %d\n", q[front++]);
    if (front > rear) front = rear = -1;
}
void peek(void) {
    if (front == -1) printf("Queue is empty\n");
    else printf("Front: %d\n", q[front]);
}
void display(void) {
    int i;
    if (front == -1) { printf("Queue is empty\n"); return; }
    for (i = front; i <= rear; i++) printf("%d ", q[i]);
    printf("\n");
}
int main(void) {
    int ch;
    do {
        printf("\n1.Insert 2.Delete 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
```

```
        switch (ch) {
            case 1: insert(); break; case 2: deleteItem(); break;
            case 3: peek(); break; case 4: display(); break;
            case 5: break; default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}
```

Q7. Dynamic Queue using a Resizable Array

A circular resizable array is used so deleted positions can be reused.

```
#include <stdio.h>
#include <stdlib.h>
int *q, front = 0, count = 0, capacity = 2;
void insert(void) {
    int x, *temp, i;
    printf("Enter element: "); scanf("%d", &x);
    if (count == capacity) {
        temp = (int *)malloc(2 * capacity * sizeof(int));
        if (temp == NULL) { printf("Memory allocation failed\n"); return; }
        for (i = 0; i < count; i++) temp[i] = q[(front + i) % capacity];
        free(q); q = temp; capacity *= 2; front = 0;
    }
    q[(front + count) % capacity] = x; count++;
}
void deleteItem(void) {
    if (count == 0) { printf("Queue Underflow\n"); return; }
    printf("Deleted: %d\n", q[front]);
    front = (front + 1) % capacity; count--;
}
void peek(void) {
    if (count == 0) printf("Queue is empty\n");
    else printf("Front: %d\n", q[front]);
}
void display(void) {
    int i;
    if (count == 0) { printf("Queue is empty\n"); return; }
    for (i = 0; i < count; i++) printf("%d ", q[(front + i) % capacity]);
    printf("\n");
}
int main(void) {
    int ch;
    q = (int *)malloc(capacity * sizeof(int));
    if (q == NULL) return 1;
    do {
        printf("\n1.Insert 2.Delete 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: insert(); break; case 2: deleteItem(); break;
            case 3: peek(); break; case 4: display(); break;
            case 5: break; default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    free(q); return 0;
}
```

Q8. Circular Queue using an Array

```
#include <stdio.h>
#define MAX 5
int q[MAX], front = -1, rear = -1;
void insert(void) {
    int x;
    if ((rear + 1) % MAX == front) { printf("Queue Overflow\n"); return; }
    printf("Enter element: "); scanf("%d", &x);
    if (front == -1) front = rear = 0;
    else rear = (rear + 1) % MAX;
    q[rear] = x;
}
void deleteItem(void) {
    if (front == -1) { printf("Queue Underflow\n"); return; }
    printf("Deleted: %d\n", q[front]);
    if (front == rear) front = rear = -1;
    else front = (front + 1) % MAX;
}
void peek(void) {
    if (front == -1) printf("Queue is empty\n");
    else printf("Front: %d\n", q[front]);
}
void display(void) {
    int i;
```

```
    if (front == -1) { printf("Queue is empty\n"); return; }
    i = front;
    while (1) {
        printf("%d ", q[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}
int main(void) {
    int ch;
    do {
        printf("\n1.Insert 2.Delete 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: insert(); break; case 2: deleteItem(); break;
            case 3: peek(); break; case 4: display(); break;
            case 5: break; default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}
```

Q9. Queue using a Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *front = NULL, *rear = NULL;
void insert(void) {
    int x; struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    if (n == NULL) { printf("Memory allocation failed\n"); return; }
    printf("Enter element: "); scanf("%d", &x);
    n->data = x; n->next = NULL;
    if (rear == NULL) front = rear = n;
    else { rear->next = n; rear = n; }
}
void deleteItem(void) {
    struct Node *t;
    if (front == NULL) { printf("Queue Underflow\n"); return; }
    t = front; printf("Deleted: %d\n", t->data);
    front = front->next; if (front == NULL) rear = NULL; free(t);
}
void peek(void) {
    if (front == NULL) printf("Queue is empty\n");
    else printf("Front: %d\n", front->data);
}
void display(void) {
    struct Node *t = front;
    if (t == NULL) { printf("Queue is empty\n"); return; }
    while (t != NULL) { printf("%d ", t->data); t = t->next; }
    printf("\n");
}
int main(void) {
    int ch;
    do {
        printf("\n1.Insert 2.Delete 3.Peek 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: insert(); break; case 2: deleteItem(); break;
            case 3: peek(); break; case 4: display(); break;
            case 5: break; default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    while (front != NULL) { struct Node *t = front; front = front->next; free(t); }
    return 0;
}
```

Q10. Singly Linked List: Insert at Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *head = NULL;
void display(void) {
    struct Node *t = head;
    if (!t) { printf("List is empty\n"); return; }
    while (t) { printf("%d -> ", t->data); t = t->next; }
    printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n = malloc(sizeof(struct Node));
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
```

```
    n->data=x; n->next=head; head=n;
}
void insertEnd(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)), *t=head;
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=NULL;
    if (!head) { head=n; return; }
    while(t->next) t=t->next; t->next=n;
}
void deleteBeg(void) {
    struct Node *t;
    if(!head){printf("List is empty\n");return;}
    t=head; head=head->next; free(t);
}
void deleteEnd(void) {
    struct Node *t=head;
    if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next->next)t=t->next;
    free(t->next);t->next=NULL;
}
void insertAfter(void) {
    int key,x; struct Node *t=head,*n;
    printf("Insert after which value? ");scanf("%d",&key);
    while(t && t->data!=key)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t->next;t->next=n;
}
void insertBefore(void) {
    int key,x; struct Node *t=head,*p=NULL,*n;
    printf("Insert before which value? ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t;
    if(!p)head=n;else p->next=n;
}
void deleteValue(void) {
    int key; struct Node *t=head,*p=NULL;
    printf("Enter value to delete: ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    if(!p)head=t->next;else p->next=t->next;
    free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
    int ch;
    do {
        printf("\n1.Insert Beginning 2.Insert End 3.Display 0.Exit\nChoice: ");
        scanf("%d", &ch);
        switch(ch) {
            case 1: insertBeg(); break;
            case 2: insertEnd(); break;
            case 3: display(); break;
            default: if(ch != 0) printf("Invalid choice\\n");
        }
    } while(ch != 0);
    deleteAll();
    return 0;
}
```

Q11. Singly Linked List: Delete from Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *head = NULL;
void display(void) {
    struct Node *t = head;
    if (!t) { printf("List is empty\n"); return; }
    while (t) { printf("%d -> ", t->data); t = t->next; }
    printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n = malloc(sizeof(struct Node));
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=head; head=n;
```

```
}
void insertEnd(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)), *t=head;
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=NULL;
    if (!head) { head=n; return; }
    while(t->next) t=t->next; t->next=n;
}
void deleteBeg(void) {
    struct Node *t;
    if(!head){printf("List is empty\n");return;}
    t=head; head=head->next; free(t);
}
void deleteEnd(void) {
    struct Node *t=head;
    if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next->next)t=t->next;
    free(t->next);t->next=NULL;
}
void insertAfter(void) {
    int key,x; struct Node *t=head,*n;
    printf("Insert after which value? ");scanf("%d",&key);
    while(t && t->data!=key)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t->next;t->next=n;
}
void insertBefore(void) {
    int key,x; struct Node *t=head,*p=NULL,*n;
    printf("Insert before which value? ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t;
    if(!p)head=n;else p->next=n;
}
void deleteValue(void) {
    int key; struct Node *t=head,*p=NULL;
    printf("Enter value to delete: ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    if(!p)head=t->next;else p->next=t->next;
    free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
    int ch;
    do {
        printf("\n1.Delete Beginning 2.Delete End 3.Display 0.Exit\nChoice: ");
        scanf("%d", &ch);
        switch(ch) {
            case 1: deleteBeg(); break;
            case 2: deleteEnd(); break;
            case 3: display(); break;
            default: if(ch != 0) printf("Invalid choice\\n");
        }
    } while(ch != 0);
    deleteAll();
    return 0;
}
```

Q12. Singly Linked List: Insert After a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *head = NULL;
void display(void) {
    struct Node *t = head;
    if (!t) { printf("List is empty\n"); return; }
    while (t) { printf("%d -> ", t->data); t = t->next; }
    printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n = malloc(sizeof(struct Node));
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=head; head=n;
}
```

```
void insertEnd(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)), *t=head;
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=NULL;
    if (!head) { head=n; return; }
    while(t->next) t=t->next; t->next=n;
}
void deleteBeg(void) {
    struct Node *t;
    if(!head){printf("List is empty\n");return;}
    t=head; head=head->next; free(t);
}
void deleteEnd(void) {
    struct Node *t=head;
    if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next->next)t=t->next;
    free(t->next);t->next=NULL;
}
void insertAfter(void) {
    int key,x; struct Node *t=head,*n;
    printf("Insert after which value? ");scanf("%d",&key);
    while(t && t->data!=key)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t->next;t->next=n;
}
void insertBefore(void) {
    int key,x; struct Node *t=head,*p=NULL,*n;
    printf("Insert before which value? ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t;
    if(!p)head=n;else p->next=n;
}
void deleteValue(void) {
    int key; struct Node *t=head,*p=NULL;
    printf("Enter value to delete: ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    if(!p)head=t->next;else p->next=t->next;
    free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
    int ch;
    do {
        printf("\n1.Insert After 2.Delete Last 3.Display 0.Exit\nChoice: ");
        scanf("%d", &ch);
        switch(ch) {
            case 1: insertAfter(); break;
            case 2: deleteEnd(); break;
            case 3: display(); break;
            default: if(ch != 0) printf("Invalid choice\\n");
        }
    } while(ch != 0);
    deleteAll();
    return 0;
}
```

Q13. Singly Linked List: Insert Before a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *head = NULL;
void display(void) {
    struct Node *t = head;
    if (!t) { printf("List is empty\n"); return; }
    while (t) { printf("%d -> ", t->data); t = t->next; }
    printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n = malloc(sizeof(struct Node));
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=head; head=n;
}
void insertEnd(void) {
```

```
    int x; struct Node *n=malloc(sizeof(struct Node)), *t=head;
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=NULL;
    if (!head) { head=n; return; }
    while(t->next) t=t->next; t->next=n;
}
void deleteBeg(void) {
    struct Node *t;
    if(!head){printf("List is empty\n");return;}
    t=head; head=head->next; free(t);
}
void deleteEnd(void) {
    struct Node *t=head;
    if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next->next)t=t->next;
    free(t->next);t->next=NULL;
}
void insertAfter(void) {
    int key,x; struct Node *t=head,*n;
    printf("Insert after which value? ");scanf("%d",&key);
    while(t && t->data!=key)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t->next;t->next=n;
}
void insertBefore(void) {
    int key,x; struct Node *t=head,*p=NULL,*n;
    printf("Insert before which value? ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t;
    if(!p)head=n;else p->next=n;
}
void deleteValue(void) {
    int key; struct Node *t=head,*p=NULL;
    printf("Enter value to delete: ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    if(!p)head=t->next;else p->next=t->next;
    free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
    int ch;
    do {
        printf("\n1.Insert Before 2.Delete Last 3.Display 0.Exit\nChoice: ");
        scanf("%d", &ch);
        switch(ch) {
            case 1: insertBefore(); break;
            case 2: deleteEnd(); break;
            case 3: display(); break;
            default: if(ch != 0) printf("Invalid choice\\n");
        }
    } while(ch != 0);
    deleteAll();
    return 0;
}
```

Q14. Singly Linked List: Delete a Given Node and Entire List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *head = NULL;
void display(void) {
    struct Node *t = head;
    if (!t) { printf("List is empty\n"); return; }
    while (t) { printf("%d -> ", t->data); t = t->next; }
    printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n = malloc(sizeof(struct Node));
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=head; head=n;
}
void insertEnd(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)), *t=head;
```

```
    if (!n) return;
    printf("Enter value: "); scanf("%d", &x);
    n->data=x; n->next=NULL;
    if (!head) { head=n; return; }
    while(t->next) t=t->next; t->next=n;
}
void deleteBeg(void) {
    struct Node *t;
    if(!head){printf("List is empty\n");return;}
    t=head; head=head->next; free(t);
}
void deleteEnd(void) {
    struct Node *t=head;
    if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next->next)t=t->next;
    free(t->next);t->next=NULL;
}
void insertAfter(void) {
    int key,x; struct Node *t=head,*n;
    printf("Insert after which value? ");scanf("%d",&key);
    while(t && t->data!=key)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t->next;t->next=n;
}
void insertBefore(void) {
    int key,x; struct Node *t=head,*p=NULL,*n;
    printf("Insert before which value? ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter new value: ");scanf("%d",&x);
    n->data=x;n->next=t;
    if(!p)head=n;else p->next=n;
}
void deleteValue(void) {
    int key; struct Node *t=head,*p=NULL;
    printf("Enter value to delete: ");scanf("%d",&key);
    while(t && t->data!=key){p=t;t=t->next;}
    if(!t){printf("Value not found\n");return;}
    if(!p)head=t->next;else p->next=t->next;
    free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
    int ch;
    do {
        printf("\n1.Delete Given Value 2.Delete Entire List 3.Display 0.Exit\nChoice: ");
        scanf("%d", &ch);
        switch(ch) {
            case 1: deleteValue(); break;
            case 2: deleteAll(); printf("List deleted\\n"); break;
            case 3: display(); break;
            default: if(ch != 0) printf("Invalid choice\\n");
        }
    } while(ch != 0);
    deleteAll();
    return 0;
}
```

Q15. Doubly Linked List: Insert at Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev, *next; };
struct Node *head=NULL;
void display(void) {
    struct Node *t=head;
    if(!t){printf("List is empty\n");return;}
    while(t){printf("%d <-> ",t->data);t=t->next;} printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)); if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->prev=NULL;n->next=head;
    if(head)head->prev=n;head=n;
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node)),*t=head;if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->next=NULL;
```

```
    if(!head){n->prev=NULL;head=n;return;}
    while(t->next)t=t->next;t->next=n;n->prev=t;
}
void deleteBeg(void) {
    struct Node *t;if(!head){printf("List is empty\n");return;}
    t=head;head=head->next;if(head)head->prev=NULL;free(t);
}
void deleteEnd(void) {
    struct Node *t=head;if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next)t=t->next;t->prev->next=NULL;free(t);
}
void insertAfter(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert after value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->prev=t;n->next=t->next;
    if(t->next)t->next->prev=n;t->next=n;
}
void insertBefore(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert before value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->next=t;n->prev=t->prev;
    if(t->prev)t->prev->next=n;else head=n;t->prev=n;
}
void deleteValue(void) {
    int k;struct Node *t=head;
    printf("Value to delete: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    if(t->prev)t->prev->next=t->next;else head=t->next;
    if(t->next)t->next->prev=t->prev;free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert Beginning 2.Insert End 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertBeg();break;
   case 2:insertEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q16. Doubly Linked List: Delete from Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev, *next; };
struct Node *head=NULL;
void display(void) {
    struct Node *t=head;
    if(!t){printf("List is empty\n");return;}
    while(t){printf("%d <-> ",t->data);t=t->next;} printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)); if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->prev=NULL;n->next=head;
    if(head)head->prev=n;head=n;
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node)),*t=head;if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->next=NULL;
    if(!head){n->prev=NULL;head=n;return;}
    while(t->next)t=t->next;t->next=n;n->prev=t;
}
void deleteBeg(void) {
    struct Node *t;if(!head){printf("List is empty\n");return;}
    t=head;head=head->next;if(head)head->prev=NULL;free(t);
}
```

```
void deleteEnd(void) {
    struct Node *t=head;if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next)t=t->next;t->prev->next=NULL;free(t);
}
void insertAfter(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert after value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->prev=t;n->next=t->next;
    if(t->next)t->next->prev=n;t->next=n;
}
void insertBefore(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert before value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->next=t;n->prev=t->prev;
    if(t->prev)t->prev->next=n;else head=n;t->prev=n;
}
void deleteValue(void) {
    int k;struct Node *t=head;
    printf("Value to delete: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    if(t->prev)t->prev->next=t->next;else head=t->next;
    if(t->next)t->next->prev=t->prev;free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
 int ch;
 do {
  printf("\n1.Delete Beginning 2.Delete End 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:deleteBeg();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q17. Doubly Linked List: Insert After a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev, *next; };
struct Node *head=NULL;
void display(void) {
    struct Node *t=head;
    if(!t){printf("List is empty\n");return;}
    while(t){printf("%d <-> ",t->data);t=t->next;} printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)); if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->prev=NULL;n->next=head;
    if(head)head->prev=n;head=n;
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node)),*t=head;if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->next=NULL;
    if(!head){n->prev=NULL;head=n;return;}
    while(t->next)t=t->next;t->next=n;n->prev=t;
}
void deleteBeg(void) {
    struct Node *t;if(!head){printf("List is empty\n");return;}
    t=head;head=head->next;if(head)head->prev=NULL;free(t);
}
void deleteEnd(void) {
    struct Node *t=head;if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next)t=t->next;t->prev->next=NULL;free(t);
}
void insertAfter(void) {
    int k,x;struct Node *t=head,*n;
```

```
    printf("Insert after value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->prev=t;n->next=t->next;
    if(t->next)t->next->prev=n;t->next=n;
}
void insertBefore(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert before value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->next=t;n->prev=t->prev;
    if(t->prev)t->prev->next=n;else head=n;t->prev=n;
}
void deleteValue(void) {
    int k;struct Node *t=head;
    printf("Value to delete: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    if(t->prev)t->prev->next=t->next;else head=t->next;
    if(t->next)t->next->prev=t->prev;free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert After 2.Delete Last 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertAfter();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q18. Doubly Linked List: Insert Before a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev, *next; };
struct Node *head=NULL;
void display(void) {
    struct Node *t=head;
    if(!t){printf("List is empty\n");return;}
    while(t){printf("%d <-> ",t->data);t=t->next;} printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)); if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->prev=NULL;n->next=head;
    if(head)head->prev=n;head=n;
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node)),*t=head;if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->next=NULL;
    if(!head){n->prev=NULL;head=n;return;}
    while(t->next)t=t->next;t->next=n;n->prev=t;
}
void deleteBeg(void) {
    struct Node *t;if(!head){printf("List is empty\n");return;}
    t=head;head=head->next;if(head)head->prev=NULL;free(t);
}
void deleteEnd(void) {
    struct Node *t=head;if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next)t=t->next;t->prev->next=NULL;free(t);
}
void insertAfter(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert after value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->prev=t;n->next=t->next;
    if(t->next)t->next->prev=n;t->next=n;
```

```
}
void insertBefore(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert before value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->next=t;n->prev=t->prev;
    if(t->prev)t->prev->next=n;else head=n;t->prev=n;
}
void deleteValue(void) {
    int k;struct Node *t=head;
    printf("Value to delete: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    if(t->prev)t->prev->next=t->next;else head=t->next;
    if(t->next)t->next->prev=t->prev;free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert Before 2.Delete Last 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertBefore();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q19. Doubly Linked List: Delete a Given Node and Entire List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev, *next; };
struct Node *head=NULL;
void display(void) {
    struct Node *t=head;
    if(!t){printf("List is empty\n");return;}
    while(t){printf("%d <-> ",t->data);t=t->next;} printf("NULL\n");
}
void insertBeg(void) {
    int x; struct Node *n=malloc(sizeof(struct Node)); if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->prev=NULL;n->next=head;
    if(head)head->prev=n;head=n;
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node)),*t=head;if(!n)return;
    printf("Enter value: ");scanf("%d",&x);
    n->data=x;n->next=NULL;
    if(!head){n->prev=NULL;head=n;return;}
    while(t->next)t=t->next;t->next=n;n->prev=t;
}
void deleteBeg(void) {
    struct Node *t;if(!head){printf("List is empty\n");return;}
    t=head;head=head->next;if(head)head->prev=NULL;free(t);
}
void deleteEnd(void) {
    struct Node *t=head;if(!head){printf("List is empty\n");return;}
    if(!head->next){free(head);head=NULL;return;}
    while(t->next)t=t->next;t->prev->next=NULL;free(t);
}
void insertAfter(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert after value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->prev=t;n->next=t->next;
    if(t->next)t->next->prev=n;t->next=n;
}
void insertBefore(void) {
    int k,x;struct Node *t=head,*n;
    printf("Insert before value: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    n=malloc(sizeof(struct Node));if(!n)return;
```

```
    printf("New value: ");scanf("%d",&x);
    n->data=x;n->next=t;n->prev=t->prev;
    if(t->prev)t->prev->next=n;else head=n;t->prev=n;
}
void deleteValue(void) {
    int k;struct Node *t=head;
    printf("Value to delete: ");scanf("%d",&k);
    while(t&&t->data!=k)t=t->next;
    if(!t){printf("Value not found\n");return;}
    if(t->prev)t->prev->next=t->next;else head=t->next;
    if(t->next)t->next->prev=t->prev;free(t);
}
void deleteAll(void) {
    while(head){struct Node *t=head;head=head->next;free(t);}
}

int main(void) {
 int ch;
 do {
  printf("\n1.Delete Given Value 2.Delete Entire List 3.Display 0.Exit\nChoice:
    ");scanf("%d",&ch);
  switch(ch) {
   case 1:deleteValue();break;
   case 2:deleteAll();printf("List deleted\\n");break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q20. Singly Circular Linked List: Insert at Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *tail=NULL;
void display(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    t=tail->next;
    do{printf("%d -> ",t->data);t=t->next;}while(t!=tail->next);
    printf("(back to head)\n");
}
void insertBeg(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;}
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;tail=n;}
}
void deleteBeg(void) {
    struct Node *h;
    if(!tail){printf("List is empty\n");return;}
    h=tail->next;
    if(h==tail)tail=NULL;else tail->next=h->next;
    free(h);
}
void deleteEnd(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    if(tail->next==tail){free(tail);tail=NULL;return;}
    t=tail->next;while(t->next!=tail)t=t->next;
    t->next=tail->next;free(tail);tail=t;
}
void insertAfter(void) {
    int k,x;struct Node *t; if(!tail)return;
    t=tail->next;printf("Insert after value: ");scanf("%d",&k);
    do{if(t->data==k)break;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t->next;t->next=n;
     if(t==tail)tail=n;}
}
void insertBefore(void) {
    int k,x;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Insert before value: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
```

```
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;p->next=n;}
}
void deleteValue(void) {
    int k;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Value to delete: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    if(t==p)tail=NULL;
    else{p->next=t->next;if(t==tail)tail=p;}
    free(t);
}
void deleteAll(void) {
    if(!tail)return;
    while(tail)deleteBeg();
}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert Beginning 2.Insert End 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertBeg();break;
   case 2:insertEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q21. Singly Circular Linked List: Delete from Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *tail=NULL;
void display(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    t=tail->next;
    do{printf("%d -> ",t->data);t=t->next;}while(t!=tail->next);
    printf("(back to head)\n");
}
void insertBeg(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;}
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;tail=n;}
}
void deleteBeg(void) {
    struct Node *h;
    if(!tail){printf("List is empty\n");return;}
    h=tail->next;
    if(h==tail)tail=NULL;else tail->next=h->next;
    free(h);
}
void deleteEnd(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    if(tail->next==tail){free(tail);tail=NULL;return;}
    t=tail->next;while(t->next!=tail)t=t->next;
    t->next=tail->next;free(tail);tail=t;
}
void insertAfter(void) {
    int k,x;struct Node *t; if(!tail)return;
    t=tail->next;printf("Insert after value: ");scanf("%d",&k);
    do{if(t->data==k)break;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t->next;t->next=n;
     if(t==tail)tail=n;}
}
void insertBefore(void) {
    int k,x;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Insert before value: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
```

```
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;p->next=n;}
}
void deleteValue(void) {
    int k;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Value to delete: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    if(t==p)tail=NULL;
    else{p->next=t->next;if(t==tail)tail=p;}
    free(t);
}
void deleteAll(void) {
    if(!tail)return;
    while(tail)deleteBeg();
}

int main(void) {
 int ch;
 do {
  printf("\n1.Delete Beginning 2.Delete End 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:deleteBeg();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q22. Singly Circular Linked List: Insert After a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *tail=NULL;
void display(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    t=tail->next;
    do{printf("%d -> ",t->data);t=t->next;}while(t!=tail->next);
    printf("(back to head)\n");
}
void insertBeg(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;}
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;tail=n;}
}
void deleteBeg(void) {
    struct Node *h;
    if(!tail){printf("List is empty\n");return;}
    h=tail->next;
    if(h==tail)tail=NULL;else tail->next=h->next;
    free(h);
}
void deleteEnd(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    if(tail->next==tail){free(tail);tail=NULL;return;}
    t=tail->next;while(t->next!=tail)t=t->next;
    t->next=tail->next;free(tail);tail=t;
}
void insertAfter(void) {
    int k,x;struct Node *t; if(!tail)return;
    t=tail->next;printf("Insert after value: ");scanf("%d",&k);
    do{if(t->data==k)break;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t->next;t->next=n;
     if(t==tail)tail=n;}
}
void insertBefore(void) {
    int k,x;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Insert before value: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
```

```
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;p->next=n;}
}
void deleteValue(void) {
    int k;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Value to delete: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    if(t==p)tail=NULL;
    else{p->next=t->next;if(t==tail)tail=p;}
    free(t);
}
void deleteAll(void) {
    if(!tail)return;
    while(tail)deleteBeg();
}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert After 2.Delete Last 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertAfter();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q23. Singly Circular Linked List: Insert Before a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *tail=NULL;
void display(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    t=tail->next;
    do{printf("%d -> ",t->data);t=t->next;}while(t!=tail->next);
    printf("(back to head)\n");
}
void insertBeg(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;}
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;tail=n;}
}
void deleteBeg(void) {
    struct Node *h;
    if(!tail){printf("List is empty\n");return;}
    h=tail->next;
    if(h==tail)tail=NULL;else tail->next=h->next;
    free(h);
}
void deleteEnd(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    if(tail->next==tail){free(tail);tail=NULL;return;}
    t=tail->next;while(t->next!=tail)t=t->next;
    t->next=tail->next;free(tail);tail=t;
}
void insertAfter(void) {
    int k,x;struct Node *t; if(!tail)return;
    t=tail->next;printf("Insert after value: ");scanf("%d",&k);
    do{if(t->data==k)break;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t->next;t->next=n;
     if(t==tail)tail=n;}
}
void insertBefore(void) {
    int k,x;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Insert before value: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
```

```
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;p->next=n;}
}
void deleteValue(void) {
    int k;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Value to delete: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    if(t==p)tail=NULL;
    else{p->next=t->next;if(t==tail)tail=p;}
    free(t);
}
void deleteAll(void) {
    if(!tail)return;
    while(tail)deleteBeg();
}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert Before 2.Delete Last 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertBefore();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q24. Singly Circular Linked List: Delete a Given Node and Entire List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
struct Node *tail=NULL;
void display(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    t=tail->next;
    do{printf("%d -> ",t->data);t=t->next;}while(t!=tail->next);
    printf("(back to head)\n");
}
void insertBeg(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;}
}
void insertEnd(void) {
    int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
    printf("Enter value: ");scanf("%d",&x);n->data=x;
    if(!tail){tail=n;n->next=n;}
    else{n->next=tail->next;tail->next=n;tail=n;}
}
void deleteBeg(void) {
    struct Node *h;
    if(!tail){printf("List is empty\n");return;}
    h=tail->next;
    if(h==tail)tail=NULL;else tail->next=h->next;
    free(h);
}
void deleteEnd(void) {
    struct Node *t;
    if(!tail){printf("List is empty\n");return;}
    if(tail->next==tail){free(tail);tail=NULL;return;}
    t=tail->next;while(t->next!=tail)t=t->next;
    t->next=tail->next;free(tail);tail=t;
}
void insertAfter(void) {
    int k,x;struct Node *t; if(!tail)return;
    t=tail->next;printf("Insert after value: ");scanf("%d",&k);
    do{if(t->data==k)break;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t->next;t->next=n;
     if(t==tail)tail=n;}
}
void insertBefore(void) {
    int k,x;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Insert before value: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
```

```
     printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;p->next=n;}
}
void deleteValue(void) {
    int k;struct Node *p,*t;
    if(!tail){printf("List is empty\n");return;}
    p=tail;t=tail->next;printf("Value to delete: ");scanf("%d",&k);
    do{if(t->data==k)break;p=t;t=t->next;}while(t!=tail->next);
    if(t->data!=k){printf("Value not found\n");return;}
    if(t==p)tail=NULL;
    else{p->next=t->next;if(t==tail)tail=p;}
    free(t);
}
void deleteAll(void) {
    if(!tail)return;
    while(tail)deleteBeg();
}

int main(void) {
 int ch;
 do {
  printf("\n1.Delete Given Value 2.Delete Entire List 3.Display 0.Exit\nChoice:
    ");scanf("%d",&ch);
  switch(ch) {
   case 1:deleteValue();break;
   case 2:deleteAll();printf("List deleted\\n");break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q25. Doubly Circular Linked List: Insert at Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
struct Node *head=NULL,*tail=NULL;
void display(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}
 t=head;do{printf("%d <-> ",t->data);t=t->next;}while(t!=head);
 printf("(back to head)\n");
}
void insertBeg(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->next=head;n->prev=tail;head->prev=n;tail->next=n;head=n;}
}
void insertEnd(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->prev=tail;n->next=head;tail->next=n;head->prev=n;tail=n;}
}
void deleteBeg(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}t=head;
 if(head==tail)head=tail=NULL;
 else{head=head->next;head->prev=tail;tail->next=head;}free(t);
}
void deleteEnd(void) {
 struct Node *t;if(!tail){printf("List is empty\n");return;}t=tail;
 if(head==tail)head=tail=NULL;
 else{tail=tail->prev;tail->next=head;head->prev=tail;}free(t);
}
void insertAfter(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert after value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->prev=t;n->next=t->next;
 t->next->prev=n;t->next=n;if(t==tail)tail=n;}
}
void insertBefore(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert before value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;n->prev=t->prev;
 t->prev->next=n;t->prev=n;if(t==head)head=n;}
}
void deleteValue(void) {
 int k;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Value to delete: ");scanf("%d",&k);t=head;
```

```
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 if(head==tail)head=tail=NULL;
 else{t->prev->next=t->next;t->next->prev=t->prev;
      if(t==head)head=t->next;if(t==tail)tail=t->prev;}
 free(t);
}
void deleteAll(void){while(head)deleteBeg();}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert Beginning 2.Insert End 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertBeg();break;
   case 2:insertEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q26. Doubly Circular Linked List: Delete from Beginning and End

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
struct Node *head=NULL,*tail=NULL;
void display(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}
 t=head;do{printf("%d <-> ",t->data);t=t->next;}while(t!=head);
 printf("(back to head)\n");
}
void insertBeg(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->next=head;n->prev=tail;head->prev=n;tail->next=n;head=n;}
}
void insertEnd(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->prev=tail;n->next=head;tail->next=n;head->prev=n;tail=n;}
}
void deleteBeg(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}t=head;
 if(head==tail)head=tail=NULL;
 else{head=head->next;head->prev=tail;tail->next=head;}free(t);
}
void deleteEnd(void) {
 struct Node *t;if(!tail){printf("List is empty\n");return;}t=tail;
 if(head==tail)head=tail=NULL;
 else{tail=tail->prev;tail->next=head;head->prev=tail;}free(t);
}
void insertAfter(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert after value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->prev=t;n->next=t->next;
 t->next->prev=n;t->next=n;if(t==tail)tail=n;}
}
void insertBefore(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert before value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;n->prev=t->prev;
 t->prev->next=n;t->prev=n;if(t==head)head=n;}
}
void deleteValue(void) {
 int k;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Value to delete: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 if(head==tail)head=tail=NULL;
 else{t->prev->next=t->next;t->next->prev=t->prev;
      if(t==head)head=t->next;if(t==tail)tail=t->prev;}
 free(t);
}
void deleteAll(void){while(head)deleteBeg();}
```

```
int main(void) {
 int ch;
 do {
  printf("\n1.Delete Beginning 2.Delete End 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:deleteBeg();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q27. Doubly Circular Linked List: Insert After a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
struct Node *head=NULL,*tail=NULL;
void display(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}
 t=head;do{printf("%d <-> ",t->data);t=t->next;}while(t!=head);
 printf("(back to head)\n");
}
void insertBeg(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->next=head;n->prev=tail;head->prev=n;tail->next=n;head=n;}
}
void insertEnd(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->prev=tail;n->next=head;tail->next=n;head->prev=n;tail=n;}
}
void deleteBeg(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}t=head;
 if(head==tail)head=tail=NULL;
 else{head=head->next;head->prev=tail;tail->next=head;}free(t);
}
void deleteEnd(void) {
 struct Node *t;if(!tail){printf("List is empty\n");return;}t=tail;
 if(head==tail)head=tail=NULL;
 else{tail=tail->prev;tail->next=head;head->prev=tail;}free(t);
}
void insertAfter(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert after value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->prev=t;n->next=t->next;
 t->next->prev=n;t->next=n;if(t==tail)tail=n;}
}
void insertBefore(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert before value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;n->prev=t->prev;
 t->prev->next=n;t->prev=n;if(t==head)head=n;}
}
void deleteValue(void) {
 int k;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Value to delete: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 if(head==tail)head=tail=NULL;
 else{t->prev->next=t->next;t->next->prev=t->prev;
      if(t==head)head=t->next;if(t==tail)tail=t->prev;}
 free(t);
}
void deleteAll(void){while(head)deleteBeg();}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert After 2.Delete Last 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertAfter();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
```

```
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q28. Doubly Circular Linked List: Insert Before a Given Node and Delete Last

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
struct Node *head=NULL,*tail=NULL;
void display(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}
 t=head;do{printf("%d <-> ",t->data);t=t->next;}while(t!=head);
 printf("(back to head)\n");
}
void insertBeg(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->next=head;n->prev=tail;head->prev=n;tail->next=n;head=n;}
}
void insertEnd(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->prev=tail;n->next=head;tail->next=n;head->prev=n;tail=n;}
}
void deleteBeg(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}t=head;
 if(head==tail)head=tail=NULL;
 else{head=head->next;head->prev=tail;tail->next=head;}free(t);
}
void deleteEnd(void) {
 struct Node *t;if(!tail){printf("List is empty\n");return;}t=tail;
 if(head==tail)head=tail=NULL;
 else{tail=tail->prev;tail->next=head;head->prev=tail;}free(t);
}
void insertAfter(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert after value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->prev=t;n->next=t->next;
 t->next->prev=n;t->next=n;if(t==tail)tail=n;}
}
void insertBefore(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert before value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;n->prev=t->prev;
 t->prev->next=n;t->prev=n;if(t==head)head=n;}
}
void deleteValue(void) {
 int k;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Value to delete: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 if(head==tail)head=tail=NULL;
 else{t->prev->next=t->next;t->next->prev=t->prev;
      if(t==head)head=t->next;if(t==tail)tail=t->prev;}
 free(t);
}
void deleteAll(void){while(head)deleteBeg();}

int main(void) {
 int ch;
 do {
  printf("\n1.Insert Before 2.Delete Last 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
  switch(ch) {
   case 1:insertBefore();break;
   case 2:deleteEnd();break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q29. Doubly Circular Linked List: Delete a Given Node and Entire List

```c
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev,*next; };
struct Node *head=NULL,*tail=NULL;
void display(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}
 t=head;do{printf("%d <-> ",t->data);t=t->next;}while(t!=head);
 printf("(back to head)\n");
}
void insertBeg(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->next=head;n->prev=tail;head->prev=n;tail->next=n;head=n;}
}
void insertEnd(void) {
 int x;struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("Enter value: ");scanf("%d",&x);n->data=x;
 if(!head){head=tail=n;n->next=n->prev=n;}
 else{n->prev=tail;n->next=head;tail->next=n;head->prev=n;tail=n;}
}
void deleteBeg(void) {
 struct Node *t;if(!head){printf("List is empty\n");return;}t=head;
 if(head==tail)head=tail=NULL;
 else{head=head->next;head->prev=tail;tail->next=head;}free(t);
}
void deleteEnd(void) {
 struct Node *t;if(!tail){printf("List is empty\n");return;}t=tail;
 if(head==tail)head=tail=NULL;
 else{tail=tail->prev;tail->next=head;head->prev=tail;}free(t);
}
void insertAfter(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert after value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->prev=t;n->next=t->next;
 t->next->prev=n;t->next=n;if(t==tail)tail=n;}
}
void insertBefore(void) {
 int k,x;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Insert before value: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 {struct Node *n=malloc(sizeof(struct Node));if(!n)return;
 printf("New value: ");scanf("%d",&x);n->data=x;n->next=t;n->prev=t->prev;
 t->prev->next=n;t->prev=n;if(t==head)head=n;}
}
void deleteValue(void) {
 int k;struct Node *t;if(!head){printf("List is empty\n");return;}
 printf("Value to delete: ");scanf("%d",&k);t=head;
 do{if(t->data==k)break;t=t->next;}while(t!=head);
 if(t->data!=k){printf("Value not found\n");return;}
 if(head==tail)head=tail=NULL;
 else{t->prev->next=t->next;t->next->prev=t->prev;
      if(t==head)head=t->next;if(t==tail)tail=t->prev;}
 free(t);
}
void deleteAll(void){while(head)deleteBeg();}

int main(void) {
 int ch;
 do {
  printf("\n1.Delete Given Value 2.Delete Entire List 3.Display 0.Exit\nChoice:
    ");scanf("%d",&ch);
  switch(ch) {
   case 1:deleteValue();break;
   case 2:deleteAll();printf("List deleted\\n");break;
   case 3:display();break;
   default: if(ch!=0)printf("Invalid choice\\n");
  }
 }while(ch!=0);
 deleteAll();return 0;
}
```

Q30. Quick Sort using Recursion

```c
#include <stdio.h>
void quickSort(int a[], int low, int high) {
    int i=low,j=high,pivot=a[(low+high)/2],temp;
    while(i<=j) {
        while(a[i]<pivot)i++;
        while(a[j]>pivot)j--;
        if(i<=j){temp=a[i];a[i]=a[j];a[j]=temp;i++;j--;}
    }
```

```
    if(low<j)quickSort(a,low,j);
    if(i<high)quickSort(a,i,high);
}
int main(void) {
    int a[100],n,i;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");
    for(i=0;i<n;i++)scanf("%d",&a[i]);
    quickSort(a,0,n-1);
    printf("Sorted array: ");
    for(i=0;i<n;i++)printf("%d ",a[i]);
    printf("\n");return 0;
}
```

Q31. Merge Sort

```
#include <stdio.h>
void merge(int a[],int l,int m,int r) {
    int temp[100],i=l,j=m+1,k=0,x;
    while(i<=m&&j<=r)temp[k++]=(a[i]<=a[j])?a[i++]:a[j++];
    while(i<=m)temp[k++]=a[i++];
    while(j<=r)temp[k++]=a[j++];
    for(x=0;x<k;x++)a[l+x]=temp[x];
}
void mergeSort(int a[],int l,int r) {
    int m;if(l<r){m=(l+r)/2;mergeSort(a,l,m);mergeSort(a,m+1,r);merge(a,l,m,r);}
}
int main(void) {
    int a[100],n,i;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);
    mergeSort(a,0,n-1);
    printf("Sorted array: ");for(i=0;i<n;i++)printf("%d ",a[i]);
    printf("\n");return 0;
}
```

Q32. Insertion Sort using a Function

```
#include <stdio.h>
void insertionSort(int a[],int n) {
    int i,key,j;
    for(i=1;i<n;i++){key=a[i];j=i-1;
        while(j>=0&&a[j]>key){a[j+1]=a[j];j--;}
        a[j+1]=key;
    }
}
int main(void) {
    int a[100],n,i;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);
    insertionSort(a,n);
    printf("Sorted array: ");for(i=0;i<n;i++)printf("%d ",a[i]);
    printf("\n");return 0;
}
```

Q33. Selection Sort using a Function (Display after Every Pass)

```
#include <stdio.h>
void selectionSort(int a[],int n) {
    int i,j,min,temp,k;
    for(i=0;i<n-1;i++){
        min=i;
        for(j=i+1;j<n;j++)if(a[j]<a[min])min=j;
        temp=a[i];a[i]=a[min];a[min]=temp;
        printf("Pass %d: ",i+1);
        for(k=0;k<n;k++)printf("%d ",a[k]);
        printf("\n");
    }
}
int main(void) {
    int a[100],n,i;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);
    selectionSort(a,n);return 0;
}
```

Q34. Bubble Sort (Display after Every Pass)

```
#include <stdio.h>
void bubbleSort(int a[],int n) {
    int i,j,temp,k,swapped;
    for(i=0;i<n-1;i++){
        swapped=0;
        for(j=0;j<n-i-1;j++)if(a[j]>a[j+1]){
            temp=a[j];a[j]=a[j+1];a[j+1]=temp;swapped=1;
        }
        printf("Pass %d: ",i+1);
        for(k=0;k<n;k++)printf("%d ",a[k]);
        printf("\n");
        if(!swapped)break;
    }
}
int main(void) {
    int a[100],n,i;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);
    bubbleSort(a,n);return 0;
}
```

Q35. Linear Search using a Function

```
#include <stdio.h>
int linearSearch(int a[],int n,int key) {
    int i;for(i=0;i<n;i++)if(a[i]==key)return i;
    return -1;
}
int main(void) {
    int a[100],n,i,key,pos;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);
    printf("Enter element to search: ");scanf("%d",&key);
    pos=linearSearch(a,n,key);
    if(pos==-1)printf("Element not found\n");
    else printf("Element found at position %d\n",pos+1);
    return 0;
}
```

Q36. Binary Search using a Function

```
#include <stdio.h>
int binarySearch(int a[],int n,int key) {
    int low=0,high=n-1,mid;
    while(low<=high){
        mid=(low+high)/2;
        if(a[mid]==key)return mid;
        if(a[mid]<key)low=mid+1;else high=mid-1;
    }
    return -1;
}
int main(void) {
    int a[100],n,i,key,pos;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements in ascending order: ");
    for(i=0;i<n;i++)scanf("%d",&a[i]);
    printf("Enter element to search: ");scanf("%d",&key);
    pos=binarySearch(a,n,key);
    if(pos==-1)printf("Element not found\n");
    else printf("Element found at position %d\n",pos+1);
    return 0;
}
```

Q37. Menu-driven Linear Search and Binary Search

Binary search is performed only after sorting the array in ascending order.

```
#include <stdio.h>
void sortArray(int a[],int n) {
    int i,j,t;
    for(i=0;i<n-1;i++)for(j=0;j<n-i-1;j++)
        if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}
}
int linearSearch(int a[],int n,int key) {
    int i;for(i=0;i<n;i++)if(a[i]==key)return i;return -1;
```

```
}
int binarySearch(int a[],int n,int key) {
    int l=0,r=n-1,m;
    while(l<=r){m=(l+r)/2;if(a[m]==key)return m;
        if(a[m]<key)l=m+1;else r=m-1;}
    return -1;
}
int main(void) {
    int a[100],n,i,ch,key,pos;
    printf("Enter number of elements: ");scanf("%d",&n);
    if(n<1||n>100)return 1;
    printf("Enter elements: ");for(i=0;i<n;i++)scanf("%d",&a[i]);
    do {
        printf("\n1.Linear Search 2.Binary Search 3.Display Sorted Array 0.Exit\nChoice: ");
        scanf("%d",&ch);
        if(ch==1||ch==2){
            printf("Enter key: ");scanf("%d",&key);
            if(ch==1)pos=linearSearch(a,n,key);
            else{sortArray(a,n);pos=binarySearch(a,n,key);}
            if(pos==-1)printf("Element not found\n");
            else printf("Element found at position %d in %s
    array\n",pos+1,ch==1?"original":"sorted");
        } else if(ch==3) {
            sortArray(a,n);for(i=0;i<n;i++)printf("%d ",a[i]);printf("\n");
        } else if(ch!=0)printf("Invalid choice\n");
    }while(ch!=0);
    return 0;
}
```

Q38. Binary Tree with Preorder, Inorder and Postorder Traversals

The program builds a Binary Search Tree by insertion; the three traversals are then performed on that binary tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left,*right; };
struct Node *createNode(int x) {
    struct Node *n=malloc(sizeof(struct Node));
    if(!n)return NULL;
    n->data=x;n->left=n->right=NULL;return n;
}
struct Node *insert(struct Node *root,int x) {
    if(root==NULL)return createNode(x);
    if(x<root->data)root->left=insert(root->left,x);
    else if(x>root->data)root->right=insert(root->right,x);
    return root;
}
void preorder(struct Node *r) {
    if(r){printf("%d ",r->data);preorder(r->left);preorder(r->right);}
}
void inorder(struct Node *r) {
    if(r){inorder(r->left);printf("%d ",r->data);inorder(r->right);}
}
void postorder(struct Node *r) {
    if(r){postorder(r->left);postorder(r->right);printf("%d ",r->data);}
}
void freeTree(struct Node *r) {
    if(r){freeTree(r->left);freeTree(r->right);free(r);}
}
int main(void) {
    struct Node *root=NULL;int n,x,i;
    printf("Enter number of nodes: ");scanf("%d",&n);
    for(i=0;i<n;i++){printf("Enter value: ");scanf("%d",&x);root=insert(root,x);}
    printf("Preorder: ");preorder(root);
    printf("\nInorder: ");inorder(root);
    printf("\nPostorder: ");postorder(root);printf("\n");
    freeTree(root);return 0;
}
```

Q39. Depth First Search (DFS) using an Adjacency Matrix

Vertices not reachable from the chosen starting vertex are not visited by this DFS run.

```
#include <stdio.h>
#define MAX 20
int adj[MAX][MAX],visited[MAX],n;
void dfs(int v) {
    int i;visited[v]=1;printf("%d ",v);
    for(i=0;i<n;i++)if(adj[v][i]&&!visited[i])dfs(i);
}
int main(void) {
    int i,j,start;
    printf("Enter number of vertices (max %d): ",MAX);scanf("%d",&n);
```

```
    if(n<1||n>MAX)return 1;
    printf("Enter adjacency matrix:\n");
    for(i=0;i<n;i++)for(j=0;j<n;j++)scanf("%d",&adj[i][j]);
    printf("Enter starting vertex (0 to %d): ",n-1);scanf("%d",&start);
    if(start<0||start>=n)return 1;
    for(i=0;i<n;i++)visited[i]=0;
    printf("DFS traversal: ");dfs(start);printf("\n");
    return 0;
}
```

Q40. Linked List to Store Details of Five Students

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct student {
    int rno;
    char name[20];
    struct student *next;
};
int main(void) {
    struct student *head=NULL,*tail=NULL,*n,*t;
    int i;
    for(i=0;i<5;i++) {
        n=(struct student *)malloc(sizeof(struct student));
        if(n==NULL){printf("Memory allocation failed\n");break;}
        printf("Enter roll number: ");scanf("%d",&n->rno);
        printf("Enter name: ");scanf("%19s",n->name);
        n->next=NULL;
        if(head==NULL)head=tail=n;
        else{tail->next=n;tail=n;}
    }
    printf("\nStudent Records\n");
    for(t=head;t!=NULL;t=t->next)
        printf("Roll No: %d  Name: %s\n",t->rno,t->name);
    while(head!=NULL){t=head;head=head->next;free(t);}
    return 0;
}
```

Q41. Student Database Management System using a Dynamic Linked List

A dynamic linked list is suitable because it supports a variable number of student records without a fixed array capacity.

```
#include <stdio.h>
#include <stdlib.h>
struct Student { int rno; char name[50]; float marks; struct Student *next; };
struct Student *head=NULL;
void addStudent(void) {
    struct Student *n=malloc(sizeof(struct Student)),*t=head;
    if(!n){printf("Memory allocation failed\n");return;}
    printf("Enter roll number: ");scanf("%d",&n->rno);
    printf("Enter name (single word): ");scanf("%49s",n->name);
    printf("Enter marks: ");scanf("%f",&n->marks);n->next=NULL;
    if(!head)head=n;
    else{while(t->next)t=t->next;t->next=n;}
}
void display(void) {
    struct Student *t=head;
    if(!t){printf("No records\n");return;}
    while(t){printf("Roll: %d Name: %s Marks: %.2f\n",t->rno,t->name,t->marks);t=t->next;}
}
void deleteStudent(void) {
    int r;struct Student *t=head,*p=NULL;
    printf("Enter roll number to delete: ");scanf("%d",&r);
    while(t&&t->rno!=r){p=t;t=t->next;}
    if(!t){printf("Record not found\n");return;}
    if(!p)head=t->next;else p->next=t->next;
    free(t);printf("Record deleted\n");
}
int main(void) {
    int ch;
    do {
        printf("\n1.Add 2.Delete 3.Display 0.Exit\nChoice: ");scanf("%d",&ch);
        switch(ch){case 1:addStudent();break;case 2:deleteStudent();break;
            case 3:display();break;case 0:break;default:printf("Invalid choice\n");}
    }while(ch!=0);
    while(head){struct Student *t=head;head=head->next;free(t);}
    return 0;
}
```